

KognitoAI — Arquitectura Cognitiva

Propuesta de Cierre de Brechas: Memoria y Razonamiento KAI

Plan de Acción Técnico para la Maduración de la Arquitectura Cognitiva Post-Refactorización (2026)

Autor: Agente Especializado KAI OS

Fecha: 24 de Julio de 2026

Versión: 2.0 (Post-Refactorización Base)

1. Resumen Ejecutivo

Tras la exitosa implementación de la refactorización base de KAI —que introdujo procedencia de datos, soporte bi-temporal en Neo4j, memoria episódica, olvido activo y resolución de conflictos—, la arquitectura se ha consolidado como un **sistema cognitivo multi-capa avanzado**.

El objetivo de esta propuesta es abordar y cerrar las **5 brechas operativas y estructurales emergentes** identificadas en el código local para elevar la plataforma al estado del arte más exigente de 2026 en materia de seguridad, explicabilidad, retención procedimental y salud de memoria en runtime.

OBJETIVOS PRINCIPALES

- Incorporar el estrato de **Memoria Procedural** para la ejecución de playbooks.
- Implementar un modelo matemático de **Decaimiento Temporal de Confianza**.
- Garantizar **Aislamiento Multi-Tenant Estricto** y registro de auditoría.
- Exponer **Explicabilidad Traza "Why"** en el pipeline de razonamiento.
- Establecer **Telemetría y Evaluación Continua** en runtime.

IMPACTO ESPERADO

- **Cero Fuga Multi-Tenant:** Validación estricta por tenant en capa de datos.
- **Reducción de Alucinaciones:** Descarte automático de hechos obsoletos no re-confirmados.
- **Transparencia Ética:** Inspección completa del proceso de inclusión/exclusión de hechos.
- **Efectividad Operativa:** Reutilización de secuencias de herramientas exitosas.

2. Diagnóstico Post-Refactorización y Brechas Identificadas

La inspección del código local confirma la presencia de pruebas y módulos para las 6 capacidades base. No obstante, se detectaron las siguientes 5 brechas que requieren atención prioritaria:

Nº	Brecha Identificada	Estado Actual en Código	Riesgo / Limitación
1	Ausencia de Memoria Procedural	Se gestiona memoria semántica, episódica y relacional, pero no flujos o playbooks exitosos.	El agente recombina herramientas desde cero en cada tarea repetitiva sin aprender procedimientos óptimos.
2	Trust Score Estático	El <code>trust_score</code> se asigna en la extracción y permanece inalterado en Neo4j.	Información antigua no verificada mantiene la misma relevancia que datos recién confirmados.
3	Aislamiento Multi-Tenant Auditado	Filtros por <code>workspace_id</code> presentes en Cypher, pero sin enforcer unificado ni audit log.	Riesgo latente de omisión accidental de filtros en consultas complejas y falta de trazabilidad de accesos.
4	Falta de Explicabilidad ("Why" Log)	Filtrado por confianza y temporalidad en el grafo ocurre como "caja negra".	Dificultad para auditar por qué un hecho específico fue incluido o excluido de la ventana de contexto.
5	Evaluación Discontinua (Ad-Hoc)	Pipeline de medición se ejecuta mediante scripts manuales.	Inexistencia de telemetría en tiempo real sobre la degradación o salud de la memoria en producción.

3. Plan de Implementación Detallado por Fases

FASE 1: IMPLEMENTACIÓN DE MEMORIA PROCEDURAL (PROCEDURAL MEMORY)

Objetivo: Almacenar y reutilizar patrones de ejecución exitosos (secuencia de herramientas, parámetros y playbooks de resolución).

Estructura de Base de Datos (PostgreSQL):

```
CREATE TABLE procedural_memory (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id UUID NOT NULL,
  workspace_id UUID,
  task_category VARCHAR(100) NOT NULL,
  procedure_name VARCHAR(150) NOT NULL,
  steps_json JSONB NOT NULL,
  success_rate FLOAT DEFAULT 1.0,
  usage_count INT DEFAULT 1,
  last_executed_at TIMESTAMPTZ DEFAULT NOW(),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Componentes a desarrollar:

- `core/procedural_memory_manager.py` : Administra la inserción, actualización de tasa de éxito y recuperación de procedimientos.
- Inyección de guía procedimental en `unified_context_node.py` cuando la intención del usuario coincida con una categoría guardada.
- Test unitario e integración en `tests/test_procedural_memory.py` .

FASE 2: DYNAMIC TRUST DECAY (DECAIMIENTO TEMPORAL DE CONFIANZA)

Objetivo: Reducir dinámicamente la confianza de hechos en el grafo a medida que pasa el tiempo sin re-confirmación.

Modelo Matemático:

$$T_{\text{dinámico}} = T_{\text{base}} \times e^{-\lambda \times \Delta t_{\text{días}}} \quad (\lambda = 0.01 \rightarrow \text{vida media } \sim 70 \text{ días})$$

Consulta Cypher Adaptada (neo4j_adapter.py):

```
MATCH (e:Entity) WHERE e.workspace_id = $workspace_id
WITH e, duration.between(e.updated_at, datetime()).days AS days_old
WITH e, e.trust_score * exp(-0.01 * days_old) AS dynamic_trust
WHERE dynamic_trust >= $min_trust
RETURN e, dynamic_trust ORDER BY dynamic_trust DESC
```

FASE 3: AISLAMIENTO MULTI-TENANT ESTRICTO Y AUDITORÍA DE MEMORIA

Objetivo: Garantizar la imposibilidad de fuga de datos entre inquilinos y mantener un registro inmutable de operaciones sobre la memoria.

Enforcer de Aislamiento

Wrapper obligatorio en `utils/neo4j_adapter.py` que intercepta toda llamada a la base de datos. Arroja `TenantIsolationException` si no se proveen explícitamente `account_id` y `workspace_id`.

Tabla de Auditoría (SQL)

Tabla `memory_access_audit` que registra fecha, usuario, acción (READ, WRITE, DELETE, RESOLVE), componente afectado y resumen de la consulta.

FASE 4: OBSERVABILIDAD DE TRAZA Y EXPLICABILIDAD ("WHY" LOG)

Objetivo: Proveer transparencia total sobre la inclusión y exclusión de recuerdos en la generación de respuestas.

Objeto `ReasoningTrajectory` (`core/models.py`):

```
{
  "query": "...",
  "candidates_retrieved": 18,
  "filtered_out": [
    {"node_id": "e_102", "reason": "low_dynamic_trust", "score": 0.32},
    {"node_id": "e_205", "reason": "outdated_bitemporal", "valid_to": "2026-02-10"}
  ],
  "included_in_context": 5
}
```

FASE 5: TELEMETRÍA DE MEMORIA CONTINUA Y WORKER DE SALUD

Objetivo: Monitorear la integridad y calidad del grafo de conocimiento en segundo plano.

Servicio **MemoryHealthWorker** (services/memory_health_worker.py):

Tarea en segundo plano que ejecuta chequeos diarios automatizados: detección de nodos huérfanos en Neo4j, evaluación del índice de frescura temporal, cálculo del score de resistencia a poisoning y reporte continuo hacia el pipeline de medición.

4. Matriz de Priorización y Esfuerzo

Fase	Iniciativa	Urgencia	Esfuerzo	Módulo Principal
Fase 1	Memoria Procedural	Media	2 días	core/procedural_memory_manager.py
Fase 2	Trust Decay Dinámico	Alta	1 día	utils/neo4j_adapter.py
Fase 3	Aislamiento y Auditoría	Alta	2 días	core/enhanced_memory_manager.py
Fase 4	Observabilidad "Why" Log	Media	1.5 días	skills/ ... / graph_reasoning_node.py
Fase 5	Telemetría Continua	Baja	2 días	services/memory_health_worker.py

5. Criterios de Aceptación y Próximos Pasos

1. Cobertura del 100% de los nuevos componentes con suites de prueba en tests/ .
2. Ejecución de benchmark de degradación mostrando 0% de fuga inter-tenant.
3. Validación de decaimiento temporal en escenarios simulados de más de 90 días.